RX / SPORTS ANALYTICS

PGE Implementation Guide

NBA and NFL player environment models
From shared pace and game script to prop probabilities

LOOKBACK / NEW SEASONS / CODE LOGIC / PRICING

A guide to the implementation in this repository, including the exact history windows, minimum samples, seasonal carryover, chronological calibration, saved forecast behavior and limitations.

Read section 0 first for the answers to how many games are used and what happens when a new season has little or no history.

The original research PDF is retained separately as a source reference. This manual explains what the code actually implements; advanced proposals in the source papers are not represented as completed features.

Code review date: 20 October 2026
Documentation source: models/Player_Stats/Pace_Game_Environment/README.md

Contents

0. Exact lookbacks and a new season 3

1. Relationship to the research methodology 4

2. Input preparation and point-in-time controls 4

3. Learned regulation volume 5

4. Common script, overtime and role scenarios 5

5. NBA opportunity and efficiency calculation 5

6. NFL opportunity and efficiency calculation 6

7. Distribution, pricing, Edge and criteria 6

8. CLI, saved artifacts and reproducibility 7

9. Evaluation, code map and remaining limits 8

This package implements an NBA/NFL public-data baseline for the supplied `Pace_Game_Environment_Player_Props_Modeling.pdf`. The source is preserved unchanged beside the code. `PGE_Implementation_Guide.pdf`, generated from this README, explains the implementation, rather than duplicating the research PDF.

In a saved NBA or NFL FanDuel game, select **Pace & Game Environment (PGE)** in **Player Forecast Method**. Supported player selections show PGE probability, PGE fair odds and Edge. Game Outcome Forecast Method remains independent.

0. Exact lookbacks and a new season

Setting	NBA	NFL
Saved source seasons	Matchup season and previous season	Matchup season and previous season
Role window / recency half-life	20 complete appearances / 5	16 complete appearances / 3
Efficiency window / half-life	60 complete appearances / 20	32 complete appearances / 8
Team mix/role scenario window	20 complete team games	16 complete team games
Minimum player and team sample	5 complete games	3 complete games
League pace/ridge window	Last 1,024 eligible team rows	Last 512 eligible team rows
Paired residual scenario window	Last 512 games	Last 256 games
Scenario recency half-life	128 paired games	64 paired games
Chronological warmup	24 league games	24 league games
Required forward-error games after warmup	At least 24	At least 24
Pace pooling prior / ridge penalty	5 effective games / 10	5 effective games / 10
Role-kernel smoothing	25% recency-only mixture	25% recency-only mixture
Simulation count	12,000	12,000

Settings live in `config.py`. These are research defaults, not settings optimized against the displayed odds. Appearance half-lives count games, not days. Only complete, finite observations for the relevant feature family enter its window. The ridge training window counts team rows, so it normally represents twice as many rows as paired games. The chronological warmup counts unique league games, not games played by an individual team.

NBA lookup seasons use their starting year and change in October. Football lookup changes in March, retaining January/February in the preceding season. The repository loads regular season and postseason from both lookup years. There is no reset at a season boundary: last season fills a short current-season role, efficiency and team window. Current appearances progressively replace old ones; recency weights reduce older observations. League pooling stabilizes short team pace histories. This does not infer a coaching/roster change.

A rookie or newly traded player without the required complete current-club appearances remains unavailable. A veteran's older club is not used to invent current-club opportunity shares. Exact name/ID and latest known club resolution are inherited from the existing player-history adapter; ambiguous identities and histories more than 450 days stale are rejected. A new season with no saved previous season fails explicitly instead of inventing a distribution. Run the preparation command to obtain both seasons.

1. Relationship to the research methodology

The central chain in source sections 1-5 and 8 is implemented as shared game environment, team volume, player role and allocation, conversion, then a full integer outcome distribution. Pace enters the opportunity layer once. The final player per-game outcome is never multiplied by another pace adjustment.

Source concept	Implementation and scope
Own tempo plus opponent pace allowed	Lagged, pooled pace features and chronological ridge coefficients
Shared NBA regulation possessions	One common possession draw for both teams; actual OT duration sampled separately
NFL plays distinct from attempts	Designed pass, sack, scramble, other non-attempt and designed-run accounting from saved PBP
Script-dependent player exposure	Smoothly conditioned historical minute/share and team-mix scenarios using score-margin residuals
Correlated simulation	Shared paired volume, script and OT draw; finite allocation plus common QB/receiver outcomes
Probability calibration and evaluation	Forward-error volume diagnostics and frozen-quote replay with proper scores; no fitted probability recalibrator yet
Additional proposed inputs	Live injuries, routes, tracking, weather, rest/travel and neutral-situation clock features are not connected

PGE is a separate selection from UO and OAM. It does not silently compose OAM's defensive efficiency adjustments. Existing UO/OAM functions accept optional environment hooks; their own services pass no environment, preserving their existing calculation paths. NCAAF requires its own clock/scheme/prior treatment and remains disabled for PGE. College WHP remains available.

2. Input preparation and point-in-time controls

`historical_data.py` reuses the saved NBA/NFL public-source team and player adapters. NBA boxes come from the existing NBA Stats adapter. NFL boxes and PBP use existing nflverse archives. Reading a market does not fetch source history and does not request sportsbook odds. Preparation is explicit.

Both snapshot and game-start timestamps must include timezones. The feature cutoff is the earlier timestamp. An eligible result must have an event date and an availability timestamp strictly before that cutoff, with availability no earlier than the event. Conflicting game/entity duplicates are rejected. Missing availability uses the existing conservative eight-hour result proxy or next-day proxy for date-only records. Those are declared approximations, not a historical publication archive. Later statistical corrections cannot be fully reconstructed.

Team rows must form a reciprocal opponent pair with the same game ID/date. Both sides need a finite scoring margin and valid volume. Incomplete pairs are excluded; role denominators use only surviving current-club team rows. A result with a delayed availability timestamp enters a chronological training prefix only after it was available, even if its event date is earlier.

NFL PBP summaries are joined by game/team, and observed player scrambles by game/team/GSIS player ID. Accepted scrimmage types are pass, run, kneel and spike; two-point conversions, deleted events, penalties/no-play and special teams are excluded. Quarter 1-4 is regulation; later quarters are OT. Designed passes include dropbacks, passes and spikes. Pass attempts, sacks, scrambles and remaining non-attempts are mutually exclusive. Drive counts are retained for audit, but the current volume predictor directly models plays rather than drives times plays.

If PBP is absent for a saved box, the declared fallback is attempts plus carries plus sacks, with no separately observed OT or scramble split for that game. Diagnostics list source modes; the fallback is never represented as measured regulation/neutral-situation PBP. If player scramble identity is unavailable, that simulated scramble allocation remains in Other rather than being assigned to an arbitrary quoted player.

3. Learned regulation volume

`features.py` estimates NBA possessions with FGA + 0.44 FTA - OREB + TOV and averages the two teams' estimates. Team minutes divided by five give actual game duration, which must agree between sides and correspond to 48 plus five-minute OT periods. Shared regulation-equivalent volume is actual estimated possessions times 48/duration. NFL regulation volume uses accepted PBP plays through Q4; OT plays remain a separate count.

`pace_model.py` builds two lagged features for each team-game: its own recent regulation volume minus league pace, and the opponent's recent allowed volume minus league pace. Recent averages use the role half-life and team window. A team estimate is shrunk to league pace using effective_n/(effective_n + 5), where effective_n = 1/sum(normalized_weight squared).

```
predicted regulation volume = league pace + a * own deviation + b * opponent deviation
beta = solve(X' W X + ridge * I, X' W (actual_volume - league_pace))
```

Coefficients a and b are estimated from earlier targets, independently by sport. They are not copied from the source PDF's illustrative weights. The ridge has no free intercept; the rolling league mean is the baseline. Targets on the same event-date batch cannot train each other's forecasts. Prior target labels must also have been available before the prediction batch. The first eligible batch uses zero deviation coefficients until earlier training targets exist.

Each predicted batch creates held-forward residuals before its actual values enter later fits. Current parameters use all eligible targets within the capped ridge window. Forecasts are bounded below at one; there is no arbitrary upper tail clipping. Volume MAE, RMSE and bias are computed from chronological errors. These diagnostics evaluate volume, not player-prop profitability.

4. Common script, overtime and role scenarios

`game_environment.py` samples paired held-forward residual tuples, retaining both teams' volume errors, actual margins and OT. Both orientations of each historical game are considered, avoiding a lexical team-name directional bias. Scenario weights combine recency and similarity between the historical and current expected margin. Expected margin is a pooled recent net-margin proxy, not the bookmaker's spread. The Gaussian bandwidth is the historical margin SD, with a one-point lower bound.

The current scenario margin is current expected margin plus the donor game's actual-minus-expected margin. The opposite team receives its negative. NBA combines both pace predictions and paired errors into one shared regulation possession draw, then multiplies by donor duration/48. Both teams see precisely the same total possessions and duration. NFL retains each side's distinct regulation play error and donor OT-play count from the same paired game.

Team mix and player-role rows are sampled from recent history with probabilities proportional to recency_weight times [0.25 + 0.75 * margin_similarity_kernel]. Thus the same margin scenario influences multiple participants; a blowout-like scenario can favor observed blowout roles. No fixed starter-minutes haircut or invented injury flag is applied. This is a smooth empirical scenario model, not a simulated score/clock path or a validated classifier for named game scripts. Finite donor support and the kernel smoothing are explicit research assumptions.

5. NBA opportunity and efficiency calculation

The shared `Usage_Opportunity/nba_model.py` engine receives the environment hook. For each sampled historical team mix, each channel is divided by that game's paired total possessions and multiplied by the new total possessions once. Two-point attempts, three-point attempts, FTA, rebounds, assists, steals, blocks and turnovers have separate pools. FTA is not inferred from FGA.

Player participation is sampled with the margin-conditioned role. Historical minutes are converted to a regulation fraction and then scaled to this draw's actual duration. Combined modeled minutes cannot exceed five times duration; remaining minutes and opportunities belong to Other. Historical player/team shares retain role/usage variation. Conflicting sampled roles are normalized before finite multinomial allocation, so quoted-player plus Other counts equal the team pool.

Two-point, three-point and FT conversion use the existing recency-weighted beta posteriors with Jeffreys prior (0.5, 0.5). Makes cannot exceed attempts. Points equal two times made twos plus three times made threes plus made FTs. Assist pools are bounded by generated team field goals; a player's assists cannot exceed generated teammate makes. Rebounds and defensive events still use box-score pool proxies rather than shot-location/tracking chances. A PRA quote sums correlated points, rebounds and assists from the same simulation draw.

The source's example gives 102 *34/48 = 72.25 on-court possessions and 72.25* 0.28 = 20.23 FGA. Substituting its printed shooting inputs produces 20.23 *(0.78* 2 *0.48 + 0.22* 3 *0.37) + 5.2* 0.82 = 24.35239 points. The PDF prints 24.38; the implementation test uses the arithmetic result of the stated inputs. These illustrative values are not hard-coded model rates.

6. NFL opportunity and efficiency calculation

The shared NFL engine draws designed passes from projected team plays using the script-conditioned historical designed-pass share. Multinomial outcomes split those passes into attempts, sacks, scrambles and other non-attempts. Therefore attempts = designed_passes - sacks - scrambles - other. Designed runs equal plays minus designed passes; rush attempts combine designed runs and scrambles.

Observed player/team scramble shares allocate the scramble pool separately. Designed-run shares remove observed scrambles from historical carry counts. Missing player scramble mappings leave that volume in Other. This prevents distributing QB scrambles through every receiver/RB's general carry share. Gain efficiency still uses blended historical yards per carry; separate scramble gain distributions are not yet fitted.

Targets are bounded by attempts, then allocated with finite historical target shares. Routes and snaps are not inferred from boxes. Catches are beta-binomial conversions of targets. Receiving/rushing yardage uses conditional event counts, weighted yards per event and recency-weighted game residuals scaled by square root of event count. Negative yardage remains possible. QB completions exactly equal generated receiver catches across quoted players and Other; QB passing yards equal receiver yardage. Interceptions cannot exceed incompletions.

The paper's attempt example is 65 *0.59* (1 - 0.07 - 0.05 - 0.01) = 33.3645. The model estimates these rates from history rather than reusing the example. TD, anytime, first/last scorer and scorer-order markets remain unavailable: no separate red-zone/opportunity/order model is connected.

7. Distribution, pricing, Edge and criteria

All quoted players and outcomes for a saved game share simulation scenarios. `PaceForecast` stores the empirical integer probability mass, not just a mean. For Over L, win mass is Y > L, loss is Y < L, and push is Y = L; Under swaps win and loss. Half-point lines cannot push. Fair probability excludes pushes: q = win/(win + loss). If q >= 0.5, fair American odds are -100*q/(1-q); otherwise they are +100*(1-q)/q. Degenerate all-push outcomes remain unpriced.

The UI displays raw implied market probability from American odds. Edge is PGE fair probability minus that raw market probability, in percentage points. Apply Criteria uses strict greater-than comparisons for both selected thresholds across every evaluated category. Existing yellow match/red no-match behavior and Show More/Show Less remain shared with the other methods. No probability changes when criteria change. No-vig opposite-side comparisons and push-aware expected net return are also recorded when quotes permit them.

Tooltips show game volume, usage-only baseline versus PGE mean, opportunity projection, predictive interval, sample counts and latest input date. The history label beneath the dropdown includes the latest league game actually used to fit pace, not merely the last quoted-player appearance. Unavailable identities and incomplete channels show a reason

rather than fabricated probabilities. The model is market-blind: changing odds alone leaves its predictive mass unchanged. Correlated marginal forecasts must not be multiplied to price a parlay.

8. CLI, saved artifacts and reproducibility

```
python -m models.Player_Stats.Pace_Game_Environment --sport NBA --prepare --start 2026-10-25T23:00:00Z
    --teams "Boston Celtics" "Detroit Pistons"
python -m models.Player_Stats.Pace_Game_Environment --sport NFL --prepare --start 2026-10-25T17:00:00Z
    --teams "Chicago Bears" "New England Patriots"
python -m models.Player_Stats.Pace_Game_Environment --sport NBA --snapshot
    "data/market_data/your_saved_snapshot.csv" --output tmp/pge-report.json
python -m models.Player_Stats.Pace_Game_Environment --sport NFL --backtest-snapshots
    tmp/pregame-quotes.json --actuals tmp/player-actuals.csv --compare --output tmp/pge-comparison.json
```

Preparation fetches public historical sources only; it consumes no Odds API credits. It includes the existing NFL PBP preparation path. Ordinary forecast rendering reads saved inputs only. A full game snapshot is simulated once so tab changes retain consistent game outcomes. Cached forecasts and evaluation records live under `data/modeling/player_stats/pace_game_environment/`, ignored by Git. Source market CSVs remain immutable under `data/market_data`.

Cache identity includes code hashes for PGE, shared UO and relevant WHP helpers, pre-cutoff inputs, configuration, snapshot rows and cutoff. The random seed depends on sport, game ID and configured seed, not on price. Restarting restores the saved histogram; code, data or configuration changes invalidate it. Artifact diagnostics record pace coefficients, league baseline, own/opponent deviations, volume errors, OT frequency, opportunity/efficiency estimates, freshness and source mode. Forecast caches are reproducible derived data, not a permanent historical publication archive.

9. Evaluation, code map and remaining limits

`backtesting.py` reuses frozen-quote replay with strictly pregame snapshots and actual player outcomes. It reports mean error, MAE/RMSE, 90% interval coverage, Brier/reliability diagnostics, three-outcome log loss and empirical CRPS. The standalone PGE replay also compares model and raw-market binary Brier scores on identical non-push quotes. Raw odds include vig and do not identify separate push mass; that benchmark is not a de-vigged three-outcome forecast. `model_comparison.py` compares WHP, UO, OAM and PGE on the exact same game, subject, canonical market, line, side and snapshot time, while reporting skipped coverage. Do not interpret zero common quotes as a successful model comparison. No held-out profitability or calibrated tail-accuracy claim is made. There is no trained probability calibrator applied to the displayed probabilities.

The source's proposed validation buckets for injury status, clock regimes and role changes require additional as-of archives. The current tests verify chronology, shared NBA pace, OT minute budgets, one-time pace scaling, finite allocation, NFL pass/carry reconciliation, paired QB/receiver outputs, push settlement, price independence, cache restoration and dashboard behavior. These checks establish implementation consistency, not predictive superiority.

File	Responsibility
config.py	Explicit sport-specific research settings
historical_data.py / INPUT_SCHEMA.md	Saved adapters, PBP classification, identities and as-of input contract
features.py	Reciprocal pairs, NBA regulation normalization, NFL regulation/OT plays
pace_model.py	Lagged pooled features, chronological ridge and forward volume errors
game_environment.py	Shared paired scenarios, conditional roles, NBA pools and NFL non-attempt accounting
prediction_service.py	Snapshot cutoff, full-game simulation, distributions, prices and frozen artifacts
distributions.py / market_comparison.py	Push-aware predictive mass, fair odds, Edge and EV
backtesting.py / model_comparison.py	Historical replay and comparable model metrics
__main__.py	Preparation, saved-snapshot pricing and evaluation CLI

This baseline uses estimated possessions and public boxes/PBP. It does not implement player tracking, actual route participation, active injury scenarios, live weather, a clock/drive simulator, tactical matchup effects, a fully reconciled shot-by-shot NBA rebound/assist process, betting execution, or a college model. Overtime and blowout behavior are empirical historical mixtures; rare tails and new rotations require further validation on untouched chronological periods.

Build provenance

The following SHA-256 hashes identify the reviewed calculation/service files. They are documentation provenance, not a runtime cache identity.

models/Player_Stats/Pace_Game_Environment/README.md
`2e5b79702cabca2357d595ac035dad27a65d206ac802c44140f2a4fa991056d0`

models/Player_Stats/Pace_Game_Environment/__init__.py
`55e4e2da499e7d06b8147b14235d44e8363334aa0a2f37100ccb3459b4273fb2`

models/Player_Stats/Pace_Game_Environment/__main__.py
`f649dd6fc38984e1151b97efed285d6f77991cbea693d5cafd7b8988bf30ca03`

models/Player_Stats/Pace_Game_Environment/backtesting.py
`8a71a083663bd355b8713ca647135d050b65ece6cd0588244997942d0269fe5b`

models/Player_Stats/Pace_Game_Environment/config.py
`fce5bb50b2c7eab657be502de39f46b9c97c9444287133c7375e55ce5187799e`

models/Player_Stats/Pace_Game_Environment/distributions.py
`2e9c2174e661b1d7a77ea01cee98ca483801d3443b4a8bd930a15f14b3727a2e`

models/Player_Stats/Pace_Game_Environment/features.py
`ed3f879bdbf1bc9e0857cab918e2a5103ee7f207ad8cafe1e1c5ab7436bfcfb4`

models/Player_Stats/Pace_Game_Environment/game_environment.py
`115fdc083c9b64f56ece07254f3e52036d51afe3f05a5f8ae5b9a93d0c0b63f4`

models/Player_Stats/Pace_Game_Environment/historical_data.py
`227466d21b3f147c732987f0f73d6fa9793f0d9e0e0737cbb4298964c3f3537f`

models/Player_Stats/Pace_Game_Environment/market_comparison.py
`e99f4084228b541326b60647a3d03c6bddf59c06ec23b5a134cb53e2843c8de4`

models/Player_Stats/Pace_Game_Environment/model_comparison.py
`548d1d496e8a2f6ebbea47064364bb406966e6c960f29f936b5bb513d2ab9860`

models/Player_Stats/Pace_Game_Environment/pace_model.py
`98bbe94a082a168d0360aabaa22f2871f599a53d291a9b4cdbe8d0ebb6fc9081`

models/Player_Stats/Pace_Game_Environment/prediction_service.py
`8b2d0f15b37021bfa8470b1f383af5da3f7096758cc863fc0f9bab9fef13b3b4`

sports_analytics/pace_game_environment_service.py
`9ff04cf3d60c73888d1d2cd7269889ed737c0b3376530df06649617d81b546a4`

models/Player_Stats/Usage_Opportunity/nba_model.py
`73f83385fd6a84a3af0ecfc87d2574d6294fa7a0b7401522d02f9d4f093e675c`

models/Player_Stats/Usage_Opportunity/nfl_model.py
`bddfc84bf37ae3b76f11dd259b516b1eb7bec9d922e0b4bb297dc32e6bcb977c`

models/Player_Stats/Usage_Opportunity/opportunity_model.py
`50228ce5d91077d2e9677cd4cc1267232032db64d312de6844f13d7e9677f682`

models/Player_Stats/Usage_Opportunity/efficiency_model.py
`1fc93ef6b455ec782aa55f4db223fc3f7d96ddbda1e7042d0e0e40e220c075bb`

models/Player_Stats/Usage_Opportunity/distributions.py
`46a0a1b903348080016c37a482e4bfc92ffaf71a6a0ad9e72a8e59207f22a891`

models/Player_Stats/Usage_Opportunity/historical_data.py
`4a5d2b774f95c02ef8e3c3058cf7b7344103fec31e3533ad56c36656ef0dc417`